Microstructure → hardness: solution

Image regression of a synthetic hardness proxy from 256×256 grayscale micrographs (500 train / 1000 test, RMSE). Protocol: one shared RepeatedKFold(5×3, seed 42) over images for every model, so every comparison is paired by fold. Python only, no external data, no remote APIs, test images never used for training; pretrained weights only under Apache-2.0 / BSD-3 (README.md, src/weights_manifest.json).

1. Data audit

Labels are unrounded float64 generator output with no value grid (Rayleigh R ≤ 0.09 for every spacing); one PNG encoder signature for all files; no train/test shift (0/28 statistics significant after Bonferroni); IDs carry no label information (ρ = -0.010); no duplicated or near-duplicated structures.

2. Measurement bias: classic estimators track the imaging nuisance

Blur, noise and contrast vary strongly per image and are label-independent. Only 37/167 structural features correlate more with hardness than with blur (Fig. 1). The ASTM E112 mean linear intercept has ρ_y = -0.027 but ρ_{noise} = +0.756; a Hall-Petch power law gives a grain-size exponent of +0.019 instead of –0.5. Analytic blur corrections (MTF division, Richardson-Lucy, scale-space extrapolation) moved only 1 of 60 corrected features across the diagonal. 128-px crops are not label-equivalent (quadrant ICC 0.60), so all models use full images.

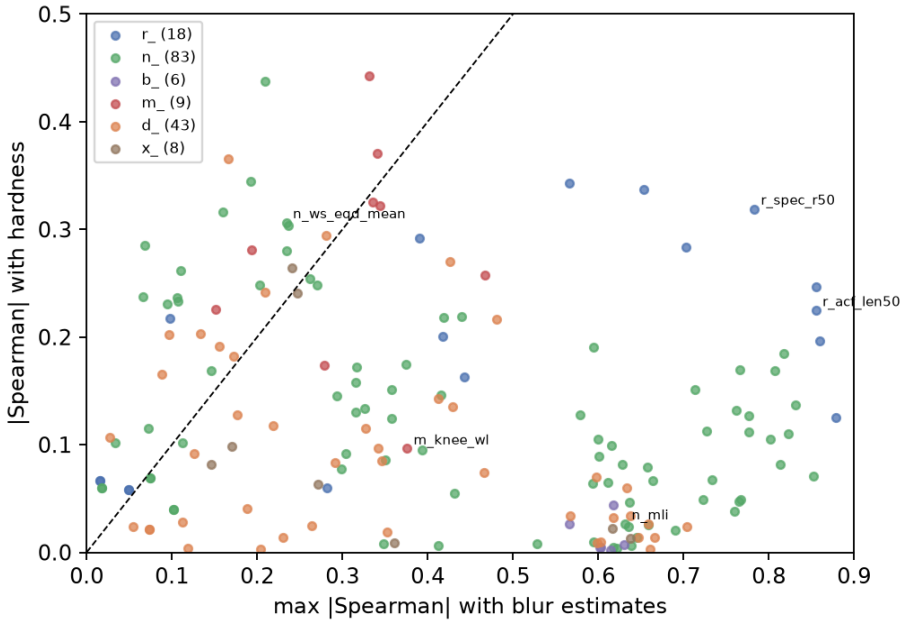


Fig. 1. One point per feature; grain-size estimators (labelled) sit far below the diagonal.

3. Signals that survived

Mode-aware dark-phase fraction. The matrix level is the KDE mode of noise-averaged grain intensities; a grain or pixel counts as dark phase when it is darker than the mode by a fixed number of gray levels (absolute thresholds beat contrast-normalized ones). **Latent net.** A ResNet18 multi-head regressor learns y-free latent measurements (dark fraction, alignment, grain size, aspect, boundary density, …) from the cleanest training images, re-degraded with blur/noise/contrast drawn from the dirty half of the training set. On held-out clean images under the heaviest degradation (dirty p90) it keeps r = 0.83 for the dark fraction where the classical estimator drops to r = 0.07. Its predictions are features of the 004 Ridge; cross-fitting them changes the CV by < 0.01.

4. CNN

timm `resnet18.a1_in1k`, in_chans=1, one output, MSE on standardized y, AdamW (lr 3e-4, wd 1e-2), 40 epochs cosine, batch 32, bf16 autocast (MPS), EMA weights, no early stopping. Full 256×256 input; augmentation D4 + Gaussian blur σ~U(0,2) + noise σ~U(0,20) (photometric augmentation hurts: contrast carries signal). OOF and test use the same inference: 8 D4 transforms, EMA weights. Screening rule for alternatives: same split (repeat 0, folds 0-4) as the baseline, drop unless clearly better.

config (repeat 0, folds 0-4)	CV RMSE	Δ vs ResNet18	fold wins
ResNet18 a1 (baseline)	13.938	–	–
ResNet18, torchvision weights	13.935	-0.003	2/5
ConvNeXt-Tiny (in22k)	13.912	-0.026	3/5
ResNet50 (tv2)	14.109	+0.172	2/5
EfficientNetV2-S (in21k)	15.898	+1.960	0/5
ResNet18, 2x upsampled input	13.579	-0.358	4/5
ResNet18, 2x upsampled, lr 1e-3	13.774	-0.163	3/5

Training RMSE without augmentation reaches 3-5 for ConvNeXt / torchvision weights while validation stays ≈ 14: the limit is generalisation from 400 images, not fit.

5. Frozen DINOv2 embeddings + Ridge

Per image (inference only): [CLS, patch-token mean, patch-token std] of the last 4 blocks of a frozen DINOv2 ViT, then a standardized RidgeCV. The candidate set {Small, Base, Large} was declared before the runs and every result is reported; 252 px (≈ 1:1 with the 256 px images) was declared the primary input size before the Base/Large runs.

input size	Small	Base	Large
126 px	14.460	14.424	14.818
168 px	13.981	14.606	14.513
224 px	13.778	14.196	14.309
252 px	13.312	13.692	14.063
448 px	13.718	13.738	13.716

Small at 252 px is the best single model of the project. Larger backbones are worse here (more dimensions than 400 training rows can support); supervised ImageNet CNN features (ConvNeXt, EfficientNetV2, ResNet50) give 14.3-14.7.

6. Final candidates

	recipe (equal-weight mean; nothing fitted on OOF)	CV RMSE	public LB	LB − CV
004	CNN ResNet18 + Ridge[features + latent net]	13.529 ± 1.010	12.3733	-1.156
005	004 + Ridge[frozen DINOv2-S]	13.113 ± 0.957	11.9317	-1.181
006	005 with the 2x-upsampled CNN (test: 5 full-data seeds)	12.988 ± 0.966	12.0015	-0.986

component	CV RMSE
CNN ResNet18 (15 folds)	13.952 ± 0.980
CNN ResNet18, 2x upsampled (15 folds)	13.669 ± 1.018
Ridge[features + latent net]	13.745 ± 1.044
Ridge[frozen DINOv2-S @252]	13.312 ± 0.882

Paired fold tests: 005 vs 004 Δ -0.416 (p 1e-05, 14/15 folds); 006 vs 004 Δ -0.542 (p 1e-06, 15/15); 006 vs 005 Δ -0.125 (p 0.002, 13/15). The CV → public-LB offset is stable for 003-005 (Fig. 2) because test predictions come from models fit on all 500 images. 006 is better than 005 on every CV view but worse on the public LB; this is unexplained, possibly public-subset noise (about six candidate comparisons on 300 public images). Ruled out: the full-data CNN models (they match the fold ensembles), ensemble size, train/test shift, and most of the selection effect.

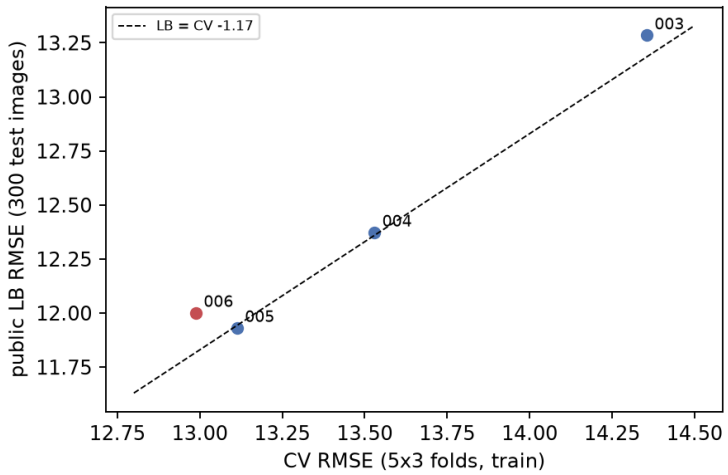


Fig. 2. CV vs public LB of the uploaded candidates.

7. Reproducibility

`python src/run.py --mode full` rebuilds 004/005/006 from the raw images (fast mode reuses the cached CNN predictions). LightGBM runs deterministic and single-threaded with sorted design matrices; PyTorch runs with fixed seeds and deterministic algorithms. In empty output folders both the fast mode and an uninterrupted full mode (all CNNs retrained, 3 h 15 min on an Apple M5) reproduce all three submissions byte-identically (SHA-256 in reports/lb_tracking.json); all CNN prediction arrays are bitwise identical. Environment, pinned versions, weight hashes and licenses: README.md. Every experiment and its CV: experiments.md.